Einführung in die Technische Informatik (ETI)

Universität Stuttgart

Wintersemester 2025/26

Inhaltsverzeichnis

1 Boolean Algebra **4**

2 VHDL Logic Operators **4**

3 CMOS Logic **5**

4 Definition of Important Terms **5**
4.1 Karnaugh Diagrams . . . 5
4.2 VHDL : Example . . . 5

5 Number representation **6**
5.1 Position Based Systems . . . 6
5.2 Decimal to Binary . . . 6
5.3 Binary to HexaDecimal . . . 6

6 Arithmetical Operations (on binary) **6**
6.1 Addition . . . 6
6.2 Subtraction . . . 6
6.3 Subtraction through Complement and Addition . . . 6
6.3.1 (B-1) Complement . . . 7
6.3.2 B's Complement . . . 7
6.3.3 Using Complements for Subtraction . . . 7
6.3.4 Representation of Negative Numbers . . . 7
6.4 Multiplication . . . 8

7 Binary Codes – Numerical Codes **8**
7.1 Binary Code . . . 8
7.2 Dual Code . . . 9
7.3 Gray Code . . . 9

8 Binary Codes – Alphanumerical Codes **10**
8.1 ASCII Code . . . 10
8.2 Unicode . . . 10

9 Binary Operations on a Hardware Level **10**
9.1 Half Adder (HA) . . . 10
9.2 Full Adder (FA) . . . 11
9.3 Ripple Carry Adder . . . 11
9.4 Binary Adder and Subtractor . . . 12

10 Floating Point Arithmetic **12**
10.1 IEEE 754 Floating Point Standard . . . 12
10.1.1 Positive Zero . . . 13
10.1.2 Negative Zero . . . 13
10.1.3 Infinities . . . 13
10.1.4 NaN – Not a Number . . . 13
10.2 Floating Point Addition and Subtraction . . . 13
10.2.1 Rounding Strategies . . . 14
10.3 Multiplication of Floating Point Numbers . . . 14

11 Sequential Circuits **14**
11.1 Dynamic Storage . . . 14
11.2 Static Storage . . . 14
11.3 Latch with Set Input . . . 15
11.4 Latch with Reset Input . . . 15
11.5 RS Latch (Reset-Set Latch) . . . 15
11.6 Gated SR Latch . . . 16
11.7 D Latch . . . 17
11.8 D Flip-Flop . . . 17
11.9 Setup and Hold Times in a D Flip-Flop . . . 17
11.10 Static Timing Analysis (STA) . . . 17
11.10.1 Timing Graph . . . 17
11.10.2 Pipelining . . . 18

12 Finite State Machines **18**
12.1 Formal Definition . . . 18
12.2 Example: Traffic Light . . . 19
12.3 Propagation Delay . . . 20

13 SRAM and DRAM **21**
13.1 RAM vs. Serial Memory . . . 21
13.1.1 Operations on RAM . . . 21
13.2 SRAM — Static RAM . . . 22
13.3 DRAM — Dynamic RAM . . . 22

14 Designing a basic computer **22**
14.1 Hardware architecture for a sum . . . 22
14.2 Hardware architecture for basic operations . . . 22
14.3 Move - MOV . . . 23
14.4 Jump - jmp . . . 23
14.4.1 Conditional jumping - bnz . . . 23
14.5 Clock speed of an ALU . . . 23
14.6 Types of computer architectures . . . 24
14.6.1 Von-Neumann-Architecture . . . 24
14.6.2 Harvard Architecture . . . 24
14.7 A computer architecture with an instruction set . . . 25
14.7.1 Definition: Computer Architecture . . . 25
14.7.2 A typical data path . . . 25
14.7.3 Instruction execution . . . 26
14.7.4 ALU - Deep Dive . . . 27
14.7.5 Shifter . . . 27
14.7.6 Barrel Shifter . . . 28
14.7.7 Register File . . . 28
14.7.8 ALU + Shifter = Function Unit . . . 29

14.7.9 Control Word . . . 29
14.7.10 Command format . . . 30
14.7.11 The Control Unit . . . 31
14.7.12 Instruction Decoder . . . 31

15 Commands **32**
15.1 Commands for Data Transfer . . . 32
15.2 Arithmetical Commands . . . 32
15.3 Logical Commands . . . 33
15.4 Control Commands . . . 33
15.5 Conditioned Jump Commands . . . 33

16 Pipelining a processor data path **33**
16.1 Pipelined Data Path . . . 33
16.1.1 Three Step Pipelining . . . 34
16.1.2 Four Step Pipelining . . . 34
16.2 Pipeline Hazards . . . 34
16.2.1 Data Hazards . . . 34
16.2.2 Control Hazards . . . 35

17 Cache and storage systems **35**
17.1 Memory hierarchy . . . 36
17.2 Principle of Locality . . . 36
17.2.1 Locality of Reference - Principle of Locality . . . 36
17.3 Cache and memory hierarchy . . . 36
17.3.1 Characteristics of the Memory Hierarchy . . . 37
17.3.2 Caches . . . 37
17.3.3 Direct Mapped Cache . . . 37
17.3.4 Fully Associative Cache . . . 37
17.3.5 Associative Memory for 4-Bit tags . . . 37

1 Boolean Algebra

It is defined as a 3-tuple $(A, +, \cdot)$, where:

- A: a set of elements
- $+, \cdot$: binary operations $A \times A \to A$

This tuple satisfies the following axioms for all $x, y, z \in A$:

1. **Commutative:** $x \cdot y = y \cdot x$ and $x + y = y + x$
2. **Distributive:** $x \cdot (y + z) = (x \cdot y) + (x \cdot z)$, and analogously for $+$
3. **Neutral Elements:** $x \cdot 1 = x, \quad x + 0 = x$
4. **Inverse Elements:** $x \cdot \overline{x} = 0, \quad x + \overline{x} = 1$

Derived Theorems

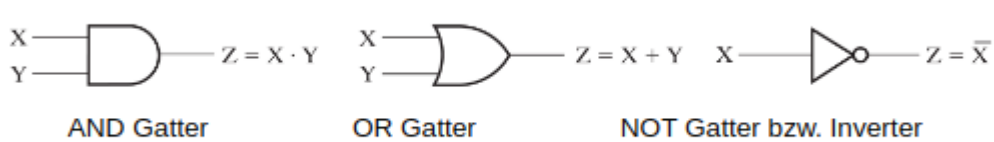
1. **Elimination:** $x \cdot 0 = 0, \quad x + 1 = 1$
2. **Absorption:** $x \cdot (x + y) = x, \quad x + (x \cdot y) = x$
3. **Associativity:** $(x \cdot y) \cdot z = x \cdot (y \cdot z)$, and analogously for $+$
4. **Idempotence:** $x \cdot x = x, \quad x + x = x$
5. **Double Negation:** $\overline{\overline{x}} = x$
6. **De Morgan:** $\overline{(x + y)} = \overline{x} \cdot \overline{y}$ and $\overline{(x \cdot y)} = \overline{x} + \overline{y}$

2 VHDL Logic Operators

VHDL is a hardware description language used to model and implement digital systems.

The expression `F <= X and Y;` assigns to `F` the result of the logical `and` between `X` and `Y`.

Logical Operator	Example in VHDL
not	`F <= not X;`
and	`F <= X and Y;`



For example, a **NAND** gate is an AND gate with a negation bubble on the output. An **XOR** gate is an OR gate with an additional curved input line. An **XNOR** gate is the XOR gate with an output bubble (inverted output).

3 CMOS Logic

CMOS (*Complementary Metal-Oxide-Semiconductor*) technology uses both N-channel and P-channel MOS transistors.

- **N-channel (NMOS):** conducts when the input voltage is HIGH; open (non-conducting) when LOW.
- **P-channel (PMOS):** conducts when the input voltage is LOW; open (non-conducting) when HIGH.

A CMOS gate consists of:

- a **Pull-Up Network (PUN)** made of PMOS transistors connected to V_{DD}
- a **Pull-Down Network (PDN)** made of NMOS transistors connected to GND

The goal is:

- when $f(a, b, \dots) = 1$, the output connects to V_{DD} through the PUN,
- when $f(a, b, \dots) = 0$, the output connects to GND through the PDN.

These networks are complementary:

$$f_n = \overline{f(a, b, \dots)}, \qquad f_p = \overline{f_n(a, b, \dots)} = f(a, b, \dots)$$

The fundamental CMOS gates are **NAND** and **NOR**; all other gates can be built from them and inverters.

4 Definition of Important Terms

- **Disjunctive Normal Form (DNF):** e.g. $abc + bcd + cde$
- **Conjunctive Normal Form (CNF / KNF):** e.g. $(a + b + c)(b + d + e)$
- **Product Term:** conjunction of literals (negations allowed)
- **Sum Term:** disjunction of literals (negations allowed)
- **Minterm:** product of all literals, each appearing exactly once; output = 1
- **Maxterm:** sum of all literals, each appearing exactly once; output = 0

4.1 Karnaugh Diagrams

Omitted for now; worth revisiting later.

4.2 VHDL : Example

```
entity compare is
  port ( a_i , b_i : in std_logic_vector ( 7 downto 0 );
    eq_o : out std_llogic );
end compare;
architecture rtl of compare is
begin
  eq_o <= '1' when ( a_i = b_i ) else '0';
end rtl;
```

5 Number representation

- Sticks: ——, ————....
- Roman Numerals, II, VI, CXL
- Position-Based-Systems

5.1 Position Based Systems

We have a basis B, and some Digits $C_i : C_i \leq B - 1, C_i \geq 0$

The value of number $Z = C_{n-1} \cdot B^{n-1} + \cdots + C_1 \cdot B^1 + C_0 \cdot B^0$

Binary system uses base 2, decimal system uses Base 10.

5.2 Decimal to Binary

Find the biggest power of 2 thats smaller than n, and subtract. Repeat until n = 0. The powers subtracted form the number.

Also possible by dividing by two and the rest is the value at that position, the coefficient is divided once again.

5.3 Binary to HexaDecimal

Grab the first 4 bits and convert to hexadecimal, repeat for the next 4 bits and so on. In case of remaining, just add zeroes.

6 Arithmetical Operations (on binary)

6.1 Addition

Its trivial, treat is as decimal addition but with the cap being 2 instead of 10.

6.2 Subtraction

Its also relatively trivial, the only trouble is with the representation of negative numbers

6.3 Subtraction through Complement and Addition

For a given subtraction $D = M - S$, we can rephrase it as:

$$D = M + (K - S) - K$$

The idea is to simplify subtraction by replacing it with addition, provided we follow some ground rules:

- $K \geq S$
- $K - S$ should be easy to compute
- K should be easy to subtract

To achieve this, we define two types of complements based on the number base B.

6.3.1 (B-1) Complement

The **(B-1) complement** of a number is obtained by subtracting each digit from $(B-1)$.

```
Example in base 10:
  Number:          352
  9's complement:  999 - 352 = 647

Example in base 2:
  Number:          10101
  1's complement:  invert all bits = 01010
```

Mathematically:

$$(B-1)\text{-complement}(N) = (B^n - 1) - N$$

where n is the number of digits (or bits) in N.

6.3.2 B's Complement

The **B's complement** of a number is obtained by adding 1 to its (B-1) complement.

```
Example in base 10:
  Number:           352
  9's complement:   647
  10's complement:  647 + 1 = 648

Example in base 2:
  Number:           10101
  1's complement:   01010
  2's complement:   01010 + 1 = 01011
```

Mathematically:

$$B\text{-complement}(N) = B^n - N = [(B-1)^n - N] + 1$$

6.3.3 Using Complements for Subtraction

To compute $M - S$ using complements:

$$M - S = M + (\text{B's complement of } S)$$

If there is an overflow (a carry beyond n digits), discard it.

```
Example in base 10 (10's complement):
  M = 352, S = 127
  10's complement of 127 = 873
  M + (10's complement of S) = 352 + 873 = 1225
  Discard overflow '1' → Result = 225
```

6.3.4 Representation of Negative Numbers

In digital systems, complements are also used to represent **negative numbers**.

1's Complement (Binary) A negative number is represented by inverting all bits of its positive value.

$$-N = 1\text{'s complement of } N$$

Example:

```
+5 = 0101
-5 = 1010   (1's complement)
```

However, this causes the issue of having both `+0 = 0000` and `-0 = 1111`, so it is rarely used in hardware.

2's Complement (Binary) In the **2's complement system**, a negative number is represented by taking the 1's complement and adding 1:

$$-N = (2\text{'s complement of } N)$$

Example:

```
+5 = 0101
1's complement = 1010
Add 1 → 1011
So, -5 = 1011 = -2*10^3 + 2 * 10^1 + 2*10^0 = -8 + 2 + 1 = -5
```

Advantages of 2's complement:

- Only one representation of zero.
- Addition and subtraction use the same circuit.
- The most significant bit (MSB) acts as a sign bit (0 = positive, 1 = negative).
- To convert the number to decimal, the leftmost digit is negative and the rest are positive.

Thus, subtraction such as $M - S$ can be done by:

$$M - S = M + (2\text{'s complement of } S)$$

and any overflow bit is discarded automatically.

6.4 Multiplication

Treat it exactly as a decimal multiplication. First digit * the number, then second digit * number * 10(one shift) and so on

7 Binary Codes – Numerical Codes

7.1 Binary Code

A **binary code** is a group of n bits that can represent 2^n different combinations of 0s and 1s. Each unique bit pattern corresponds to a specific encoded element, such as a number or character.

Example: Binary Coded Decimal (BCD) In **BCD**, each decimal digit (0–9) is represented by its own 4-bit binary code.

Binär-Codierte-Dezimalzahlen(BCD)

Dezimal symbol	BCD Symbol
0	0000
1	0001
2	0010
3	0011
4	0100
5	0101
6	0110
7	0111
8	1000
9	1001

This makes conversions between decimal and binary systems straightforward, although it is less space-efficient than pure binary representation.

7.2 Dual Code

The **dual code** (or **binary positional code**) is a standard binary representation in which each bit position has a weight of a power of 2. The numeric value of a code word is obtained by summing all the bit weights that are set to 1.

$$\text{Example: } (1011)_2 = 1 \cdot 2^3 + 0 \cdot 2^2 + 1 \cdot 2^1 + 1 \cdot 2^0 = 11_{10}$$

This is the most common numeric code used in computers for arithmetic and logic operations.

7.3 Gray Code

Gray code is a binary code designed so that consecutive numbers differ by only **one bit**. This property minimizes errors in systems where signals change gradually (e.g., rotary encoders, analog-to-digital converters).

Dual-Code	Anzahl der Bit-Wechsel	Gray-Code	Anzahl der Bit-Wechsel
000		000	
001	1	001	1
010	2	011	1
011	1	010	1
100	3	110	1
101	1	111	1
110	2	101	1
111	1	100	1
000	3	000	1

Because only one bit changes at a time, Gray code reduces the chance of misreading during transitions between states. It was used extensively in older hardware systems and remains relevant in error-sensitive applications.

8 Binary Codes – Alphanumerical Codes

8.1 ASCII Code

The **American Standard Code for Information Interchange (ASCII)** is a 7-bit character encoding system that defines 128 possible symbols.

- 94 characters are printable (letters, digits, punctuation, etc.)
- 34 are control characters (e.g., `Enter`, `Tab`, `Backspace`)

$$\text{Example: } 'A' = 65_{10} = 1000001_2$$

ASCII became the foundation for most modern text encodings and is still embedded within larger standards like Unicode.

8.2 Unicode

Because ASCII's 128-symbol limit is insufficient for all global alphabets, the **Unicode standard** was developed.

Unicode assigns a unique **code point** to every character in nearly all writing systems, symbols, and emojis. It supports over **1,000,000** possible code points.

Modern implementations typically use **UTF-8**, **UTF-16**, or **UTF-32** to store these code points efficiently.

9 Binary Operations on a Hardware Level

9.1 Half Adder (HA)

A **Half Adder** is a combinational circuit that takes two binary inputs a and b, and produces two outputs: the *sum* (s) and the *carry* (c). The sum represents $a + b$, while the carry indicates overflow when both inputs are 1.

a	b	c	s
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

It can be realized using logic gates as follows:

- $c \leftarrow a \wedge b$
- $s \leftarrow a \oplus b$

9.2 Full Adder (FA)

A **Full Adder** extends the functionality of the Half Adder by including a third input d, representing the carry from the previous digit. It produces a sum s and a carry c.

a	b	d	c	s
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

The logical computation is given by:

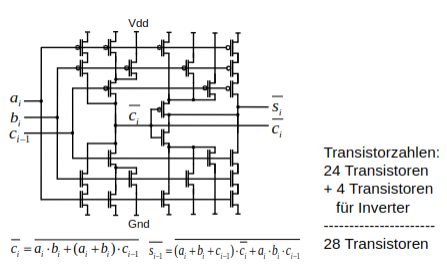
- $s \leftarrow a \oplus b \oplus d$
- $c \leftarrow (a \wedge b) \vee (a \wedge d) \vee (b \wedge d)$

Since the full adder is essential for arithmetic operations and a traditional implementation uses around 40 transistors, efforts have been made to reduce transistor count.

A key simplification arises from observing that s is typically the inverse of c, except in the cases where all inputs are either 1 or 0. Thus:

$$s_i = (a_i + b_i + d_i) \cdot \overline{c_i} + a_i \cdot b_i \cdot d_i$$

This optimized implementation requires fewer transistors.



9.3 Ripple Carry Adder

A **Ripple Carry Adder** connects multiple full adders in sequence so that the carry output of one adder becomes the carry input of the next.

VHDL code for a 4-bit ripple carry adder:

```
entity ripple_carry_adder is
  port (
    A  : in  std_logic_vector (3 downto 0);
    B  : in  std_logic_vector (3 downto 0);
    C0 : in  std_logic;
    S  : out std_logic_vector(3 downto 0);
    C4 : out std_logic
  );
end entity ripple_carry_adder;

architecture structure of ripple_carry_adder is
  signal C : std_logic_vector(4 downto 0);
begin
  C(0) <= C0;

  ripple_adder_generation : for i in 0 to 3 generate
    full_adder_I : entity work.full_adder
      port map (
        A    => A(i),
        B    => B(i),
        Cin  => C(i),
        Cout => C(i+1),
        S    => S(i)
      );
  end generate ripple_adder_generation;

  C4 <= C(4);
end architecture structure;
```

Note: A cleaner version of this VHDL code can be found in VL 4, Teil 2, page 15.

9.4 Binary Adder and Subtractor

By adding an extra input S, the circuit can function as both an adder and a subtractor:

- $S = 0$: performs addition.
- $S = 1$: performs subtraction.

For each input B_i, the value is XORed with S before entering the full adder:

$$B_i' = B_i \oplus S$$

If $S = 0$, B_i' remains unchanged. If $S = 1$, B_i' becomes $\overline{B_i}$, and by setting the initial carry input to S, the circuit adds 1, effectively computing the two's complement.

10 Floating Point Arithmetic

10.1 IEEE 754 Floating Point Standard

According to the IEEE 754 standard, a floating-point number consists of three parts:

1. **Sign bit:** The first bit; 1 represents negative, 0 represents positive.
2. **Exponent with bias:** An 8-bit unsigned value. The true exponent is obtained by subtracting the bias (127).

3. **Significand (Mantissa):** Represents the fractional part after the binary point. The integer part before the point is implicitly 1.

Example:

$$1\ 10000011\ 01000000000000000000000$$

$$-2^{131-127} \times (1.0100)_2 = -2^4 \times 1.25 = -20$$

10.1.1 Positive Zero

All bits are zero; interpreted as +0. Represents the smallest possible positive number.

10.1.2 Negative Zero

All bits are zero except for the sign bit, which is 1; interpreted as −0.

10.1.3 Infinities

Exponent is all ones, significand all zeros. Represents positive or negative infinity depending on the sign bit.

10.1.4 NaN – Not a Number

Exponent is all ones, significand is non-zero. Represents an undefined numerical result.

Zusammenfassung Fließkommazahlen

- Format: $\pm 1.bbb \ldots bb_2 \cdot 2^{ee \cdots ee}$
- Wert: $(-1)^s \cdot (1+F) \cdot 2^E$ mit $E = e - B$
- Wert: $(-1)^s \cdot (1 + M/2^p) \cdot 2^E$

Exponent E	Mantisse M	Wert	Bedeutung	Bezeichnung
$E = 0$	$M = 0$	$(-1)^S \times 0$	±0	Null (gehört zu denorm.)
$E = 0$	$M > 0$	$(-1)^S \times M / 2^p \times 2^{1-B}$	$\pm 0{,}M \times 2^{1-B}$	denormalisierte Zahl
$0 < E < 2^r-1$	$M >= 0$	$(-1)^S \times (1+M / 2^p) \times 2^{E-B}$	$\pm 1{,}M \times 2^{E-B}$	normalisierte Zahl
$E = 2^r-1$	$M = 0$	Unendlich	$\pm\infty$	Unendlich
$E = 2^r-1$	$M > 0$	keine Zahl		keine Zahl (NaN)

10.2 Floating Point Addition and Subtraction

1. **Null checking:** If one operand is zero, return the other.
2. **Exponent alignment:** Shift the significand of the smaller exponent to the right until both exponents match.
3. **Addition/Subtraction:** Perform the arithmetic on the aligned significands. Watch for overflow.
4. **Normalization:** Adjust the significand and exponent to maintain proper format.
5. **Rounding:** Apply the appropriate rounding method.

10.2.1 Rounding Strategies

Rounding Strategy	Description
Rounding towards zero	The result is truncated towards zero.
Rounding towards $\pm\infty$	The result is rounded towards positive or negative infinity, depending on the sign.
Rounding to the nearest representable number	The result is rounded to the closest representable value.

10.3 Multiplication of Floating Point Numbers

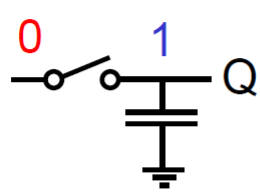
Floating-point multiplication is performed by:

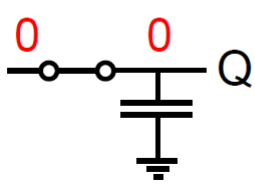
1. Adding the exponents (subtracting the bias if necessary),
2. Multiplying the significands,
3. Applying normalization and rounding.

11 Sequential Circuits

11.1 Dynamic Storage

A single transistor can be used to temporarily store a bit of data.

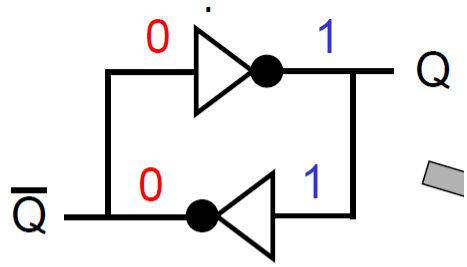


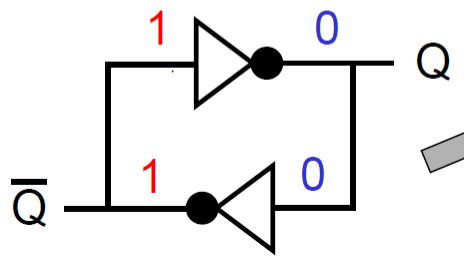


The main issue with this type of storage is that the stored voltage gradually decreases over time due to leakage currents and imperfect isolation. Therefore, it requires periodic refreshing.

11.2 Static Storage

A more reliable approach uses a **double inversion cycle**, where two inverters are connected in a feedback loop.

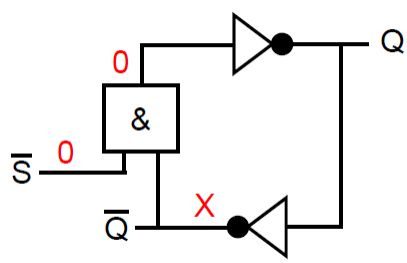




This principle forms the basis of **latches** and **flip-flops**, which are fundamental building blocks of sequential logic.

11.3 Latch with Set Input

A latch can be extended with an additional input that forces the output Q to 1 (and $\overline{Q}$ to 0). This input is called $\overline{S}$ (the active-low Set signal). When $S = 1$, then $\overline{S} = 0$ and Q becomes 1.

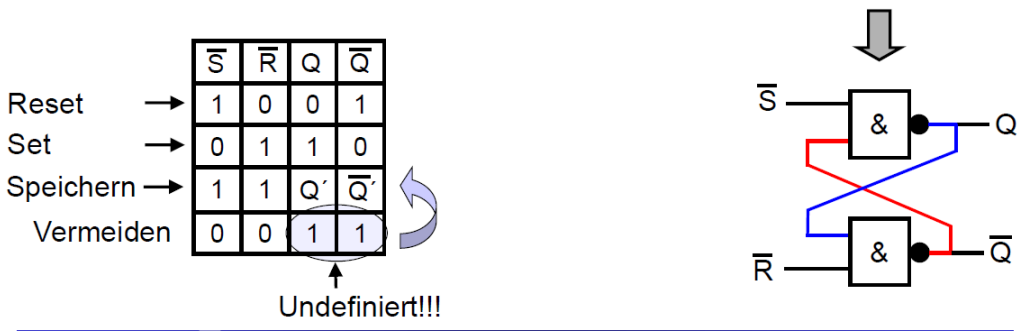


11.4 Latch with Reset Input

Similarly, a **reset input** allows the output Q to be forced to 0. The corresponding signal is $\overline{R}$ (active-low Reset).

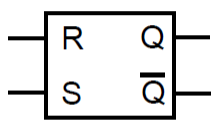
11.5 RS Latch (Reset-Set Latch)

The RS latch combines both the Set and Reset inputs.



The behavior is as follows:

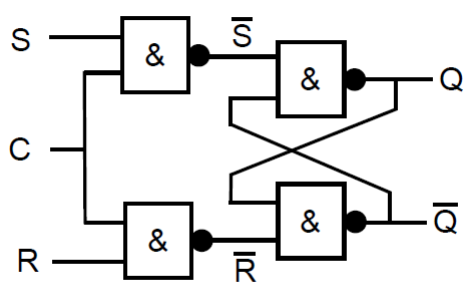
- $S = 1$, $R = 0$: Set ($Q = 1$)
- $S = 0$, $R = 1$: Reset ($Q = 0$)
- $S = R = 1$: Hold (no change)
- $S = R = 0$: Undefined state



While the RS latch is useful, it does not take the clock signal into account, which limits its reliability in synchronous systems. However, it is still suitable for simple asynchronous tasks such as key debouncing or switch state storage.

11.6 Gated SR Latch

A gated SR latch introduces a **clock (enable)** signal C that allows writing only when $C = 1$.



This implementation requires around 16 transistors.

11.7 D Latch

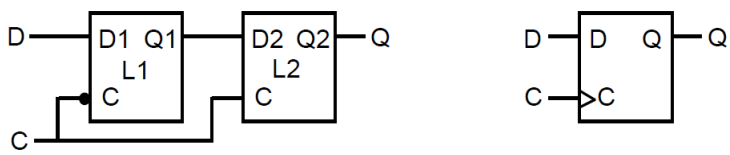
A simplified version of the SR latch is the **D latch**, which uses a single data input D and defines:

$$S := D, \qquad R := \overline{D}$$

This design requires approximately 18 transistors. Alternatively, a D latch can also be implemented using multiplexers.

The drawback of a D latch is that it is **level-sensitive**—it is transparent when the clock is high, meaning input changes immediately propagate to the output. If data is written outside the intended clock phase, unwanted behavior can occur. To address this, a second D latch is used with an inverted clock signal ($C' = \overline{C}$).

11.8 D Flip-Flop



A **D flip-flop** samples the input only at a specific clock transition (rising or falling edge), thus avoiding glitches and undefined intermediate voltages.

It can also be implemented using two D latches (a *master–slave* configuration) or multiplexers.

Issue: If the input changes too close to a clock transition, unpredictable behavior may occur. **Solution:** Define *setup* and *hold* times.

11.9 Setup and Hold Times in a D Flip-Flop

The **setup time** (t_{setup}) is the minimum time before a clock edge during which the input data must remain stable. The **hold time** (t_{hold}) is the minimum time after the clock edge during which the input must continue to remain stable.

11.10 Static Timing Analysis (STA)

Static Timing Analysis is a method used to determine the maximum operating frequency of a digital circuit without requiring simulation. It identifies the **longest propagation delay path** in the circuit — also known as the **critical path** — and uses it to calculate the highest possible clock frequency.

11.10.1 Timing Graph

A **timing graph** models a circuit as a directed graph where:

- Each node represents a logic element (gate, flip-flop, etc.) with a delay.
- Each edge represents a connection

Nodes driven by a clock signal (e.g., the output of a flip-flop) are treated as **starting points** and therefore have no incoming edges, since they mark the beginning of a new clock cycle.

The goal of STA is to ensure that, for every path between two flip-flops, the data has enough time to propagate before the next clock edge. Formally, for correct operation, the following condition must hold for every path:

$$t_{clk_q} + t_{logic} + t_{setup} \leq T_{clk}$$

where:

- t_{clk_q} is the clock-to-output delay of the source flip-flop,
- t_{logic} is the total propagation delay through the combinational logic,
- t_{setup} is the setup time of the destination flip-flop,
- T_{clk} is the clock period.

If this condition is violated, the clock frequency must be lowered until the inequality is satisfied.

11.10.2 Pipelining

Pipelining is a technique used to increase the maximum clock frequency by dividing a long combinational path into shorter segments. This is achieved by inserting **D flip-flops** between stages of the circuit.

Each flip-flop introduces a new clock boundary, effectively shortening the critical path delay for each stage:

$$T_{clk}^{new} = \max(t_{stage1}, t_{stage2}, \ldots, t_{stagen})$$

As a result, the circuit can operate at a higher clock speed, since each pipeline stage performs only a portion of the total computation per cycle. However, this also increases the **latency** (number of cycles required to complete a full operation) even though the **throughput** (operations per second) improves.

12 Finite State Machines

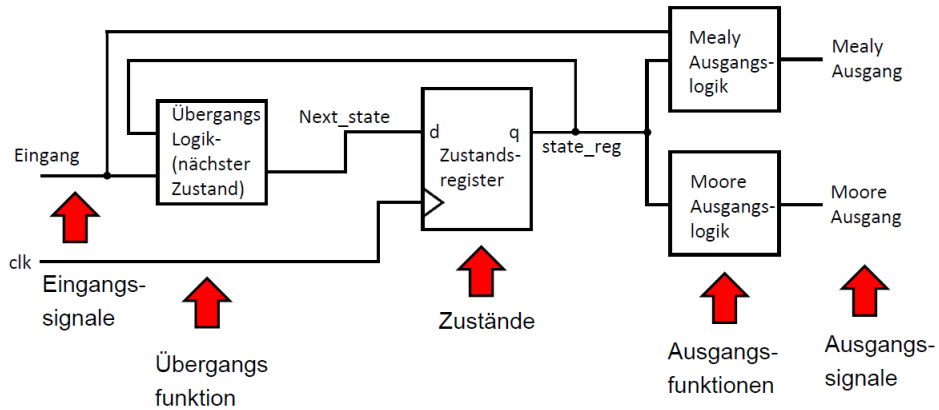
12.1 Formal Definition

A finite state transducer (FST) is defined as a 6-tuple

$$(A, S, s_0, \delta, B, \lambda),$$

where:

- A: Finite set of input symbols (alphabet)
- S: Finite set of states
- $s_0 \in S$: Starting state
- $\delta : S \times A \to S$: State transition function
- B: Finite set of output symbols
- λ: Output function
  - Mealy type: $\lambda : S \times A \to B$
  - Moore type: $\lambda : S \to B$



12.2 Example: Traffic Light

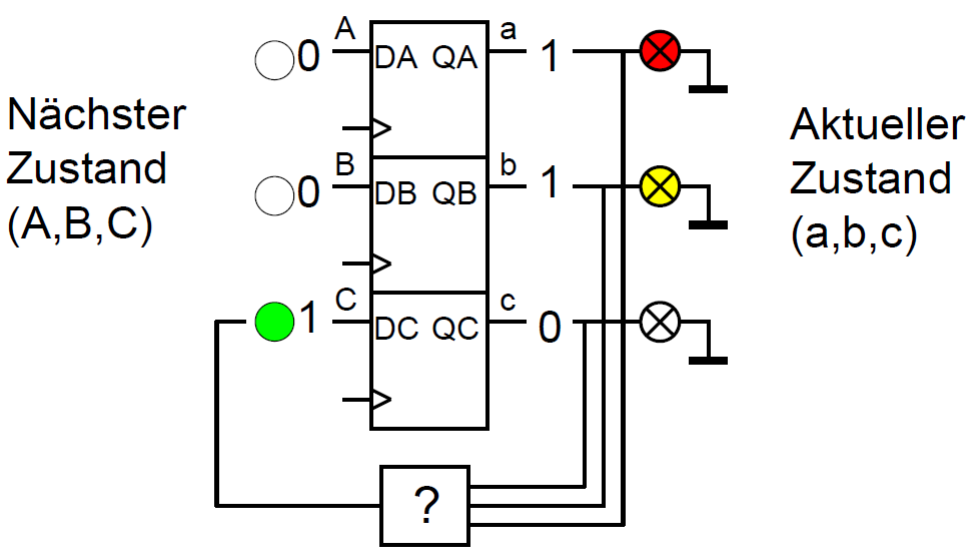
Consider a traffic light cycling through the states

$$\text{green} \to \text{yellow} \to \text{red} \to \text{red+yellow} \to \text{green}.$$

Rotating and encoding the light horizontally as binary gives:

$$001 \to 010 \to 100 \to 110 \to 001.$$

We represent the current state using a register of D-flip-flops.



We build a truth table for the transition logic:

a	b	c	A	B	C
0	0	1	0	1	0
0	1	0	1	0	0
1	0	0	1	1	0
1	1	0	0	0	1

Resulting Boolean expressions:

$$C = ab\bar{c},$$
$$B = \bar{a}\,\bar{b}\,c + a\bar{b}\,\bar{c},$$
$$A = \bar{a}\,b\bar{c} + a\bar{b}\,\bar{c}.$$

Important Note It is essential to analyze how the system behaves under invalid input states. In this example, incorrect inputs would cause the system to cycle as:

$$000 \to 000 \to 000\ldots$$

which is harmless but must be known.

12.3 Propagation Delay

We define Δ as the delay of one logic gate.

Consider the logic function

$$Z = \overline{\bar{b} + a}.$$

We switch inputs from $(a, b) = (1, 1)$ to $(0, 0)$ and track the system over time.

Before the switch ($t < 0$)

- $a = 1$
- $b = 1$
- $\bar{b} = 0$
- $Z = \overline{0 + 1} = 0$

At the moment of switching ($t = 0$)

- $a = 0$
- $b = 0$
- $\bar{b} = 0$ (not updated yet)
- $Z = 0$ (not updated yet)

After one gate delay ($\Delta < t < 2\Delta$)

- $a = 0$
- $b = 0$
- $\bar{b} = 1$ (updated)
- $Z = 0$ (updated using old inputs)

After full settling ($t > 2\Delta$)

- $a = 0$
- $b = 0$
- $\overline{b} = 1$
- $Z = \overline{1 + 0} = 1$

This results in a temporary glitch of duration Δ. In a traffic light controller, this would cause a brief flash of green after red.

Solution Place a D-flip-flop at the output to sample the value only after stabilization.

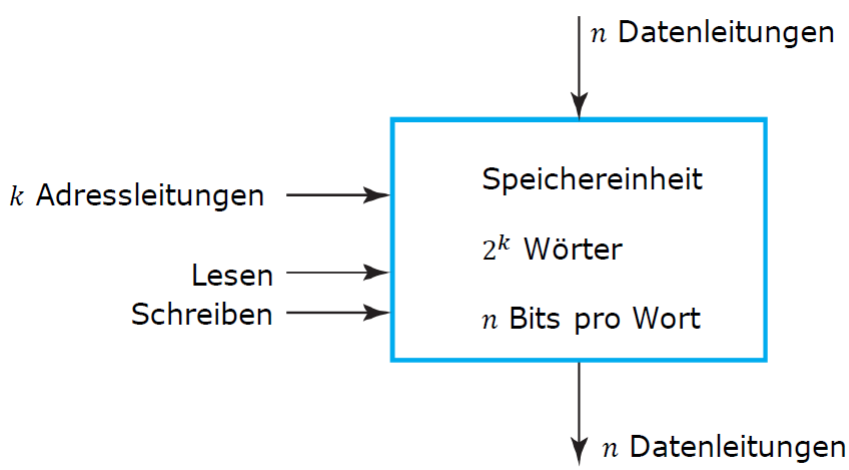
13 SRAM and DRAM

13.1 RAM vs. Serial Memory

Random Access Memory (RAM) provides constant access time for every address.

Serial memory (e.g., HDDs) has variable access time depending on mechanical factors such as disk rotation.

Word A **word** is a group of bits stored in a memory cell, typically a multiple of 8 bits (one byte).



13.1.1 Operations on RAM

- **Chip Select (CS)**: A dedicated line enabling or disabling access to the chip (1 = active, 0 = inactive).
- **Read/Write**: Determines the operation (0 = write, 1 = read). Ignored when Chip Select is 0.

13.2 SRAM — Static RAM

- Data is retained as long as power is supplied.
- Uses stable self-refreshing circuits (e.g., cross-coupled inverters).
- Faster than DRAM.
- Uses more physical area per bit.

13.3 DRAM — Dynamic RAM

- Stores data in capacitors.
- Values leak over time and must be periodically refreshed.
- Very compact: typically 1 transistor per bit.

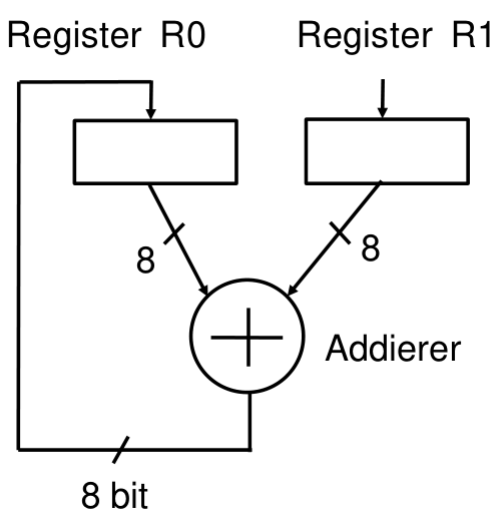
14 Designing a basic computer

14.1 Hardware architecture for a sum

Lets say we want to compute

$$y = \sum_{i=1}^{N=10} a_i$$

We can just do a circuit with a register R0 that holds the current sum, and a register R1 that starts inputting the numbers in a_i. We just combine those two into an adder, and the output feeds back to R0.



14.2 Hardware architecture for basic operations

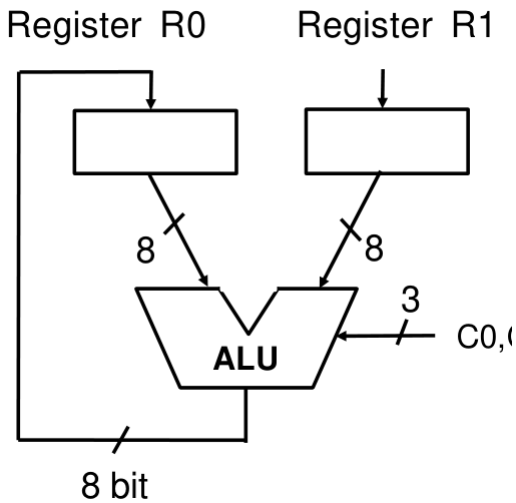
We can use a third input C to determine the operation we wish to do.

Whenever we are able to handle all the basic operations, this is called an Arithmetic Logic Unit: **ALU**

Basic operations with its C code

1. 000 add

2. 100 sub
3. 010 mul
4. 110 and
5. 001 or
6. 011 xor



Note: We also can switch the register R1 with a multiplexer and R1,..RN registers to allow all inputs from $a_1, .., a_n$

So now an instruction code will be

Operation Code (3 bits in this case) Register code (another 3 bits)

14.3 Move - MOV

Copies the contents of R0 into the set ADDRESS

14.4 Jump - jmp

Allows to jump within the assembly code.

MARK sub R2

...

jmp MARK

This code jumps back to MARK

14.4.1 Conditional jumping - bnz

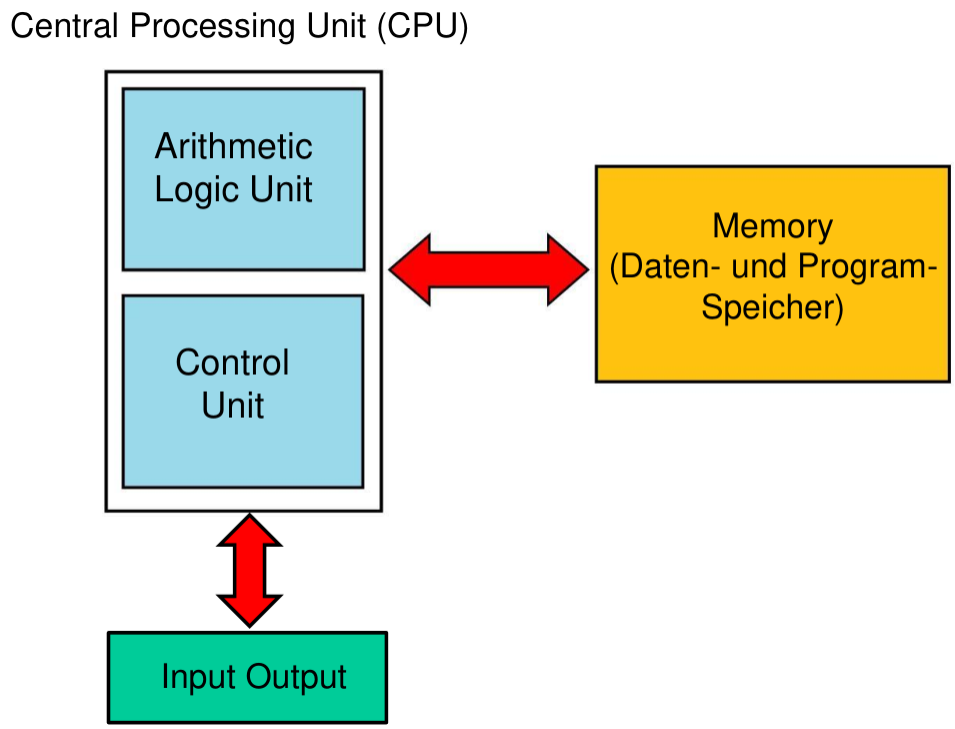
This is a conditional jump if branch not zero, meaning if the previous operation is not zero it does the jump

14.5 Clock speed of an ALU

Given that the time delay of a full adder is around 50ps, the clock speed will be around 2.5GHz

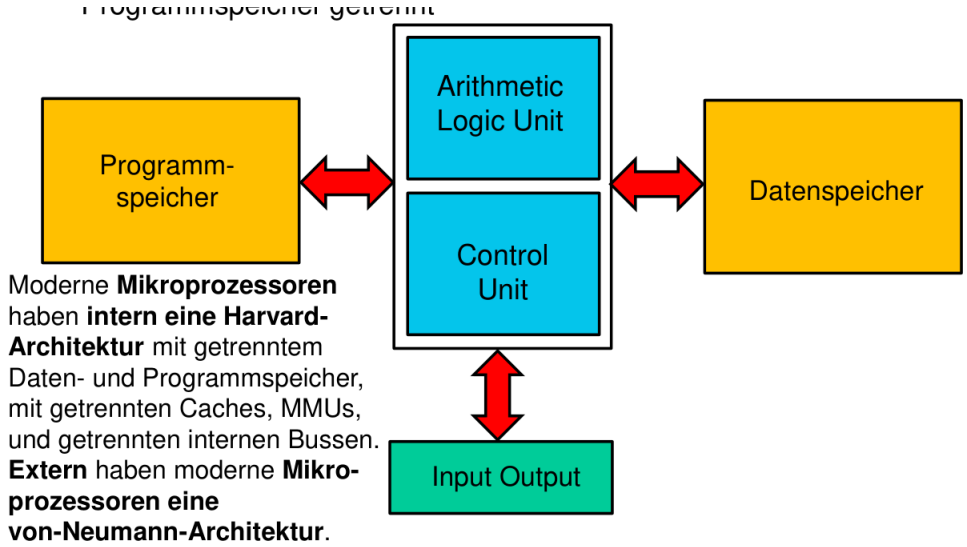
14.6 Types of computer architectures

14.6.1 Von-Neumann-Architecture



The issue with this is that the storage is shared, so in theory the code could alter itself. Another characteristic is that the words are of a set word length in this format

14.6.2 Harvard Architecture



Separated storage for programm and data, so no self altering.

14.7 A computer architecture with an instruction set

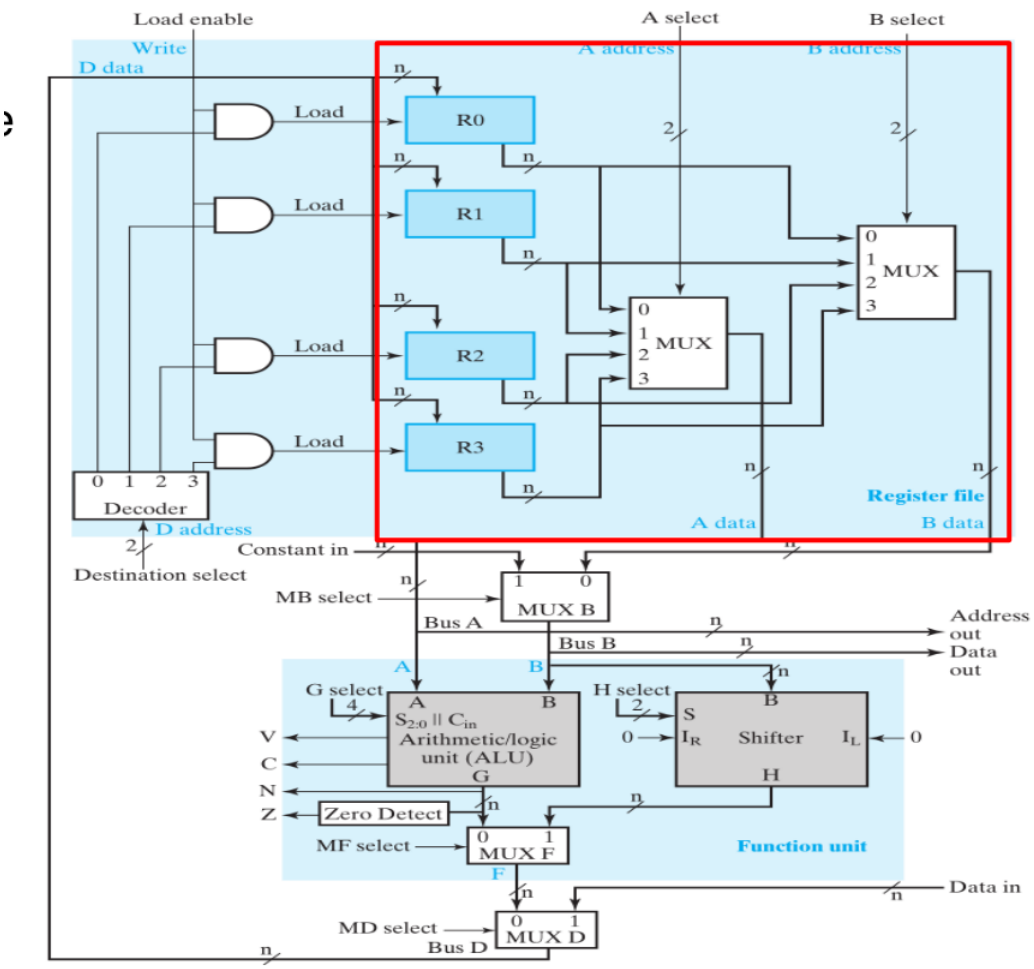
14.7.1 Definition: Computer Architecture

It can be defined as the description of the hardware at the block-diagramm level that explains the functionality of the system.

It can be divided in

- Data Path
  - ALU
  - Control interface
  - Register record
- Control unit
  - Provides signals based on the interpretation of commands

14.7.2 A typical data path



Registers: We have 4 registers in this case, with two multiplexers that allow us to select two of these registers. (A Select, B Select)

Bus A: Its also connected to the external adress lines. Used for memory operations or I/O

Bus B: Is connected to the data output. Used to send data to other components of the system that are not the data path

MUX B: Allows to fill Bus B with a constant when set set to 1

ALU: Used to do arithmetic + logic micro operations. Its inputs are the Buses A and B, with also an input called G-Select(4-bits) that selects the operation

It also contains some flags for extra info

- Z : zero status - output = 0
- C : carry status - last carry digit = 1
- V : overflow status
- N : sign status

Shifter: Used to shift bits left or right. Can only alter Bus B.

MUX F: Picks between the output of the shifter or the ALU

MUX D: picks between the output of MUX F or external data.

D Bus: Contains the data line of the input for all registers.

Destination Select: Picks which registers are filled and such with the data of the D-Bus

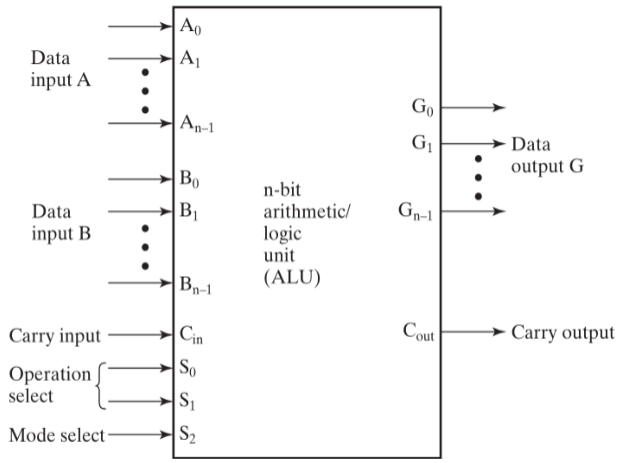
14.7.3 Instruction execution

Note: All this is done basically in parallel, but described as a list for easier understanding

Example: $R1 \leftarrow R2 + R3$

1. A Select gets the value 2 to access R2
2. B Select gets the value 3 to access R3
3. G Select gets the value for the operation $A + B$
4. MF-Select gets the value 0 to get the contents of the ALU
5. MD-Select gets the value 0 to get the contents of the Data Path
6. Destination Select picks R1 as the output Destination
7. Load Enable is set to 1 to allow the data being written into R1

14.7.4 ALU - Deep Dive



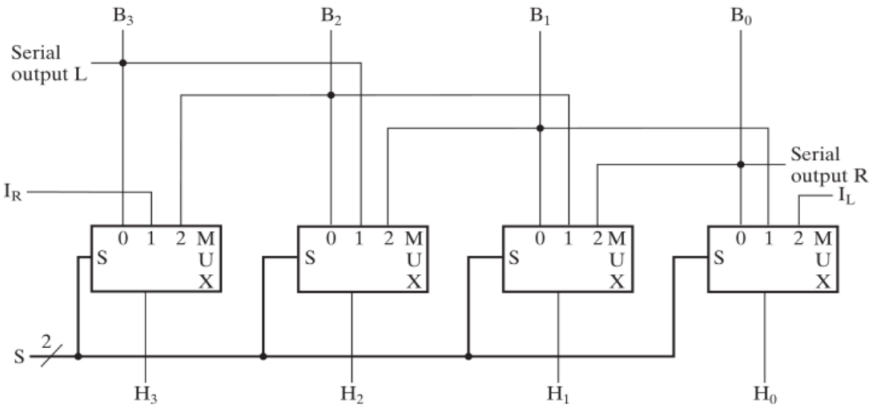
Operations on an ALU: They are the following

Operation Select					
S_2	S_1	S_0	C_{in}	Operation	Function
0	0	0	0	$G = A$	Transfer A
0	0	0	1	$G = A + 1$	Increment A
0	0	1	0	$G = A + B$	Addition
0	0	1	1	$G = A + B + 1$	Add with carry input of 1
0	1	0	0	$G = A + \overline{B}$	A plus 1s complement of B
0	1	0	1	$G = A + \overline{B} + 1$	Subtraction
0	1	1	0	$G = A - 1$	Decrement A
0	1	1	1	$G = A$	Transfer A
1	X	0	0	$G = A \wedge B$	AND
1	X	0	1	$G = A \vee B$	OR
1	X	1	0	$G = A \oplus B$	XOR
1	X	1	1	$G = \overline{A}$	NOT (1s complement)

14.7.5 Shifter

The shifter handles shifting left or right the bits of Bus B. It consists of a series of multiplexers where each multiplexer i connects to B_{i-1}, B_i, B_{i+1}. To pick the order of rotation. For the borders it contains I_R, I, L as input values. To select the type of shift you use S.

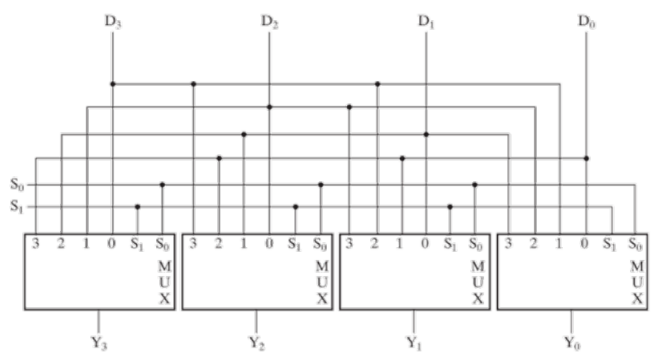
$$\begin{cases} S = 00 & \text{nothing} \\ S = 01 & \text{shift right once. } I_R \text{ is used} \\ S = 10 & \text{shift left once. } I_L \text{ is used} \end{cases}$$



14.7.6 Barrel Shifter

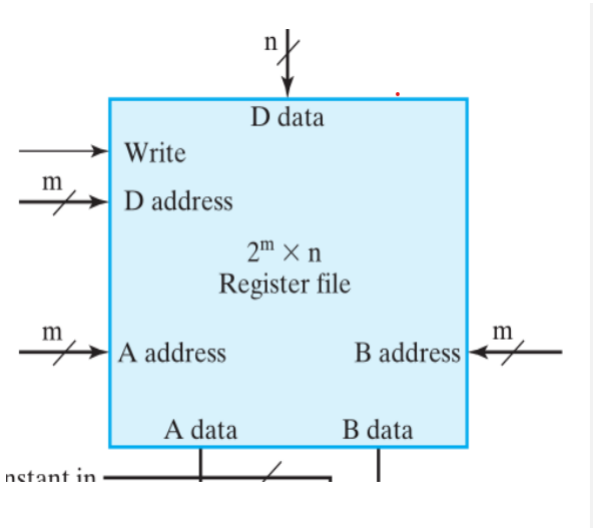
Shifts n bit positions, defined by the binary value (S_0, S_1)

Select		Output				
S_1	S_0	Y_3	Y_2	Y_1	Y_0	Operation
0	0	D_3	D_2	D_1	D_0	No rotation
0	1	D_2	D_1	D_0	D_3	Rotate one position
1	0	D_1	D_0	D_3	D_2	Rotate two positions
1	1	D_0	D_3	D_2	D_1	Rotate three positions



14.7.7 Register File

Its a fast storage comprised of registers. Allows reading/writing multiple words at at time.
m defines the amount of registers
n defines the amount of bits of the register



The grabbing the data from the A, B and D adresses is all done in the same clock cycle.

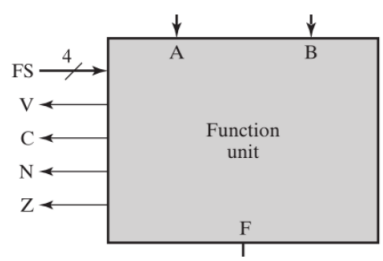
14.7.8 ALU + Shifter = Function Unit

Values:

- A,B inputs of the function
- F output
- V,C,N,Z statusbits described above
- FS 4 bits Function Select

***G* Select, *H* Select, and *MF* Select Codes Defined in Terms of *FS* Codes**

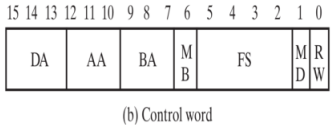
FS(3:0)	MF Select	G Select(3:0)	H Select(3:0)	Microoperation
0000	0	0000	XX	$F = A$
0001	0	0001	XX	$F = A + 1$
0010	0	0010	XX	$F = A + B$
0011	0	0011	XX	$F = A + B + 1$
0100	0	0100	XX	$F = A + \overline{B}$
0101	0	0101	XX	$F = A + \overline{B} + 1$
0110	0	0110	XX	$F = A - 1$
0111	0	0111	XX	$F = A$
1000	0	1X00	XX	$F = A \wedge B$
1001	0	1X01	XX	$F = A \vee B$
1010	0	1X10	XX	$F = A \oplus B$
1011	0	1X11	XX	$F = \overline{A}$
1100	1	XXXX	00	$F = B$
1101	1	XXXX	01	$F = \text{sr } B$
1110	1	XXXX	10	$F = \text{sl } B$



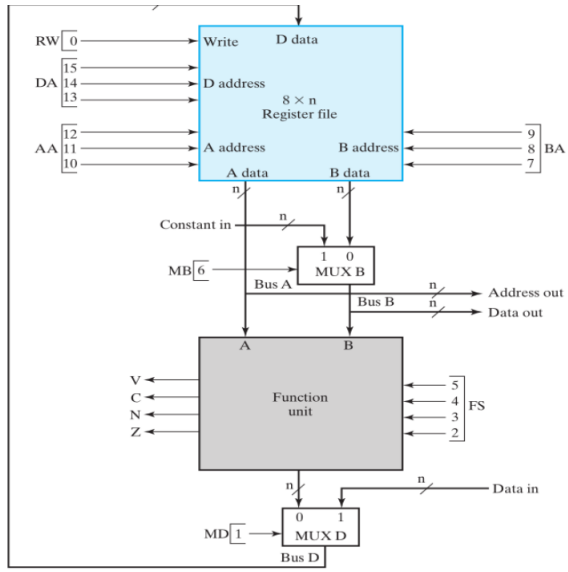
14.7.9 Control Word

We use 16 bits to define a control word for our example

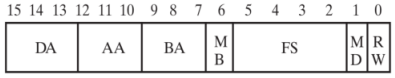
➢ Wie in der Abbildung ersichtlich, spezifizieren 16 Kontrolleingänge (Bits) ein Kontrollwort.



(b) Control word



DA, AA, BA		MB		FS		MD		RW	
Function	Code	Function	Code	Function	Code	Function	Code	Function	Code
$R0$	000	Register	0	$F = A$	0000	Function	0	No Write	0
$R1$	001	Constant	1	$F = A + 1$	0001	Data in	1	Write	1
$R2$	010			$F = A + B$	0010				
$R3$	011			$F = A + B + 1$	0011				
$R4$	100			$F = A + \overline{B}$	0100				
$R5$	101			$F = A + \overline{B} + 1$	0101				
$R6$	110			$F = A - 1$	0110				
$R7$	111			$F = A$	0111				
				$F = A \wedge B$	1000				
				$F = A \vee B$	1001				
				$F = A \oplus B$	1010				
				$F = \overline{A}$	1011				
				$F = B$	1100				
				$F = \text{sr } B$	1101				
				$F = \text{sl } B$	1110				



(b) Control word

14.7.10 Command format

Usually represented throguh a bit group

Our format is as follows

- Operation code: represents the operation to be done
- Destination register
- Source register A
- Source register B

Instruction	Opcode	Mne-monic	Format	Description	Status Bits
Move A	0000000	MOVA	RD, RA	R[DR] ← R[SA]*	N, Z
Increment	0000001	INC	RD, RA	R[DR] ← R[SA] + 1*	N, Z
Add	0000010	ADD	RD, RA, RB	R[DR] ← R[SA] + R[SB]*	N, Z
Subtract	0000101	SUB	RD, RA, RB	R[DR] ← R[SA] − R[SB]*	N, Z
Decrement	0000110	DEC	RD, RA	R[DR] ← R[SA] − 1*	N, Z
AND	0001000	AND	RD, RA, RB	R[DR] ← R[SA] ∧ R[SB]*	N, Z
OR	0001001	OR	RD, RA, RB	R[DR] ← R[SA] ∨ R[SB]*	N, Z
Exclusive OR	0001010	XOR	RD, RA, RB	R[DR] ← R[SA] ⊕ R[SB]*	N, Z
NOT	0001011	NOT	RD, RA	R[DR] ← $\overline{\text{R[SA]}}$*	N, Z
Move B	0001100	MOVB	RD, RB	R[DR] ← R[SB]*	
Shift Right	0001101	SHR	RD, RB	R[DR] ← sr R[SB]*	
Shift Left	0001110	SHL	RD, RB	R[DR] ← sl R[SB]*	
Load Immediate	1001100	LDI	RD, OP	R[DR] ← zf OP*	
Add Immediate	1000010	ADI	RD, RA, OP	R[DR] ← R[SA] + zf OP*	N, Z
Load	0010000	LD	RD, RA	R[DR] ← M[SA]*	
Store	0100000	ST	RA, RB	M[SA] ← R[SB]*	
Branch on Zero	1100000	BRZ	RA, AD	if (R[SA] = 0) PC ← PC + se AD, if (R[SA] ≠ 0) PC ← PC + 1	N, Z
Branch on Negative	1100001	BRN	RA, AD	if (R[SA] < 0) PC ← PC + se AD, if (R[SA] ≥ 0) PC ← PC + 1	N, Z
Jump	1110000	JMP	RA	PC ← R[SA]*	

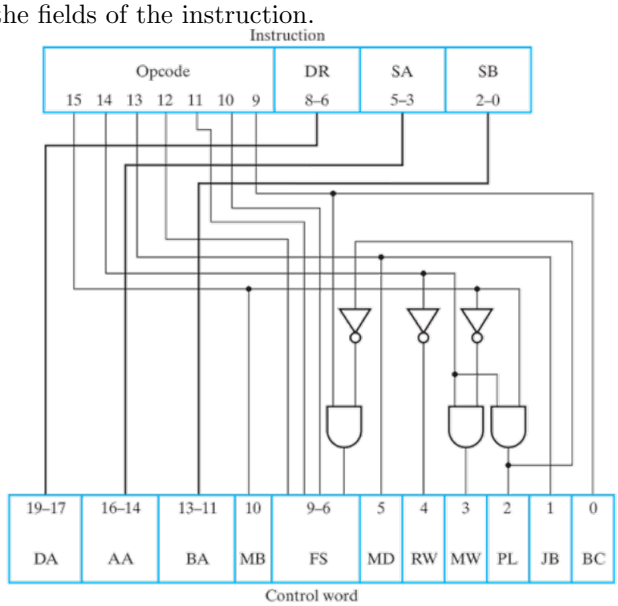
* For all of these instructions, PC ← PC + 1 is also executed to prepare for the next cycle.

14.7.11 The Control Unit

The computer delivers the command address to the command memory. The output of the command is then processed further in the decoder

14.7.12 Instruction Decoder

Combination of logic gates that provides all control words for the data path, based on the contents of the fields of the instruction.



Truth table for a Instruction Decoder: Here

Instruction Function Type	Instruction Bits				Control-Word Bits						
	15	14	13	9	MB	MD	RW	MW	PL	JB	BC
Function-unit operations using registers	0	0	0	X	0	0	1	0	0	X	X
Memory read	0	0	1	X	0	1	1	0	0	X	X
Memory write	0	1	0	X	0	X	0	1	0	X	X
Function-unit operations using register and constant	1	0	0	X	1	0	1	0	0	X	X
Conditional branch on zero (Z)	1	1	0	0	X	X	0	0	1	0	0
Conditional branch on negative (N)	1	1	0	1	X	X	0	0	1	0	1
Unconditional jump	1	1	1	X	X	X	0	0	1	1	X

15 Commands

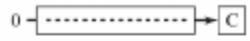
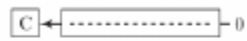
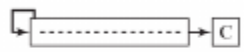
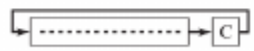
15.1 Commands for Data Transfer

Name	Mnemonic	Description
Load	LD	transfers Data from storage to a processor register
Store	ST	transfers data from processor register to a storage word
Move	MOVE	Transfers data between two registers, or between two words in storage (or one of each)
Exchange	XCH	Exchanges data between two registers or two words in storage
Push	PUSH	Stack operation
Pop	POP	Stack operation
Stack pointer	STACKPOINTER	The register that contains the adress of the stack pointer is called SP. Points to the top element of the stack
Input	IN	Moves data from/to specific registers assigned to I/O (E/A in German)
Output	OUT	Same thing, opposite

15.2 Arithmetical Commands

Name	Mnemonic
Increment	INC
Decrement	DEC
Add	ADD
Subtract	SUB
Multiply	MUL
Divide	DIV
Add with carry	ADDC
Subtract with borrow	SUBB
Subtract reverse	SUBR
Negate	NEG

15.3 Logical Commands

Name	Mnemonic	Diagram
Logical shift right	SHR	
Logical shift left	SHL	
Arithmetic shift right	SHRA	
Arithmetic shift left	SHLA	
Rotate right	ROR	
Rotate left	ROL	
Rotate right with carry	RORC	
Rotate left with carry	ROLC	

15.4 Control Commands

Name	Mnemonic
Branch	BR
Jump	JMP
Call Procedure	CALL
Return from Procedure	RET
Compare (by subtraction)	CMP
Test(by ANDing)	TEST

15.5 Conditioned Jump Commands

Uses the flags of the ALU

Branch Condition	Mnemonic	Test Condition
Branch if zero	BZ	$Z = 1$
Branch if not zero	BNZ	$Z = 0$
Branch if carry	BC	$C = 1$
Branch if no carry	BNC	$C = 0$
Branch if minus	BN	$N = 1$
Branch if plus	BNN	$N = 0$
Branch if overflow	BV	$V = 1$
Branch if no overflow	BNV	$V = 0$

Z = Zero Flag

C = Carry Flag

N = Negative Flag

V = Overflow Flag
(xor of internal carry into and out of the sign bit)

16 Pipelining a processor data path

16.1 Pipelined Data Path

This process inserts flip flops in the middle of a data path, to allow for a 'reset'. This reduces the length of the largest path (and its time), allowing for the clock to run faster, which ends up making a processor faster.

We have two important concepts regarding this

- Latency: Amount of time to process a command
- Throughput: Number of commands that can be executed in a measure of time

16.1.1 Three Step Pipelining

1. Operand Fetch (OF)
2. Execution (EX)
3. Write Back (WB)

This makes so that in a pipeline, at a certain step, there is one instruction in the OF step, another instruction in the EX step, and another in the WB step.

16.1.2 Four Step Pipelining

1. Instructin Fetch (IF)
2. Decode and Operand Fetch (DOF)
3. Execution (EX)
4. Write Back (WB)

As expected, there can be some hazards of working like this

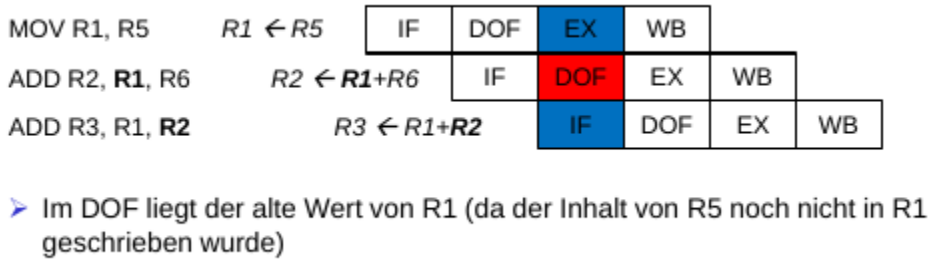
16.2 Pipeline Hazards

Hazards are problems at the clock cycle level that arise when an operation within a pipeline is delayed by one or more clock cycles relative to the time at which the instruction for the operation was fetched.

Types of hazards:

- Data hazards: occur when a subsequent instruction attempts to use the result of the operation before it has been calculated. Old values lead to incorrect results.
- Control hazards: occur during program branches.

16.2.1 Data Hazards



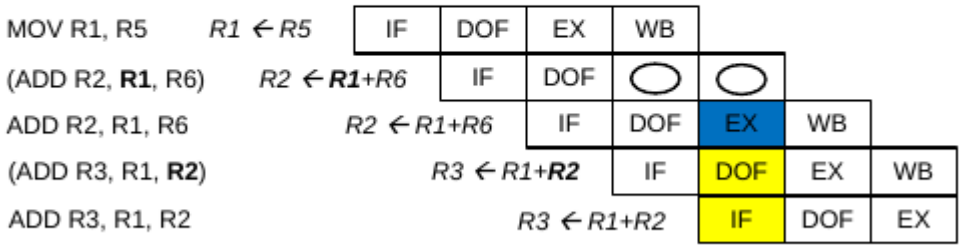
Pretty self explanatory how they happen

Software Solution: Make the compiler analize the instructions, and if we see one of those overwriting will happen, add as many NOP (No-Operation) as needed so that they act as a blank buffer to avoid this

The issues with this is that the compiler has to handle it by itself, and that it makes the code longer

Hardware Solution: If an operand is identified within the DOF stage that has not yet been written back, the associated execution of the EX phase and the write-back phase of the instruction is delayed by delaying the pipeline flow in the IF and DOF phases is delayed for one clock cycle.

- Ein weiterer Versuch einen alten Wert aus R2 zu lesen, wird mit dem zweiten ADD-Befehl durchgeführt. → Ablauf wie bei ersten ADD-Befehl.



Data Forwarding Solution: If the edge case happens, just forward the data to where it should be. Has the disatvantage of possibly reducing a bit the latency

- Weiterleitung des Wertes von R1 - der in EX berechnet wurde - direkt nach DOF

16.2.2 Control Hazards

These hazards occur when you have branching. Because you cant know previously know where the branch will jump, it can affect pipelining.

The jump will only take place in the EX step, so until the operation gets there, we are executing operations that may affect the outcome of the code.

Software Solution: Add NOP until we evaluate the branch statement. Its an extra buffer but an easy solution.

Branch Prediction Solution: Guessing which branch it will be, if it ends up not being the correct guess, we abort the execution of the instructions already put in.

17 Cache and storage systems

There is something called Moore's law, that represents that the processor performance grows faster than the memory (roughly 50

17.1 Memory hierarchy

The lowest level of the hierarchy is the smallest, fastest memory, called **Cache**

So that the hierarchy works, are a big part of the CPU-Commands and Operands fetched from the cache

$$[\text{CPU} \leftrightarrow \text{Cache}] \leftrightarrow \text{Main Memory} \leftrightarrow \text{Hard Drive}$$

The rest of the operands and CPU-Instructions are loaded from the main memory.
The cache fetches all data from the main memory
The hard drive prepares all data for the main memory.
The speed and size of memory are inversely proportional
We have two different caches

- Instruction Cache
- Data Cache

Allows to fetch an instruction and its data in a single cycle

17.2 Principle of Locality

A usual program uses Loops. These loops repeat the same instructions (but with possibly differejt data)

Example: A loop repeats 8 commands 100 times.

This produces that the fetch of each of the 8 instructions must be called 100 times. THis is called **temporal locality**, which describes the effect of having a past fetch to a memmory address that will be repeated with a high likelyhood

Apart from this, there is also a high probability that the adresses of the opeands lie in sequential order. This is called **spatial locality**

17.2.1 Locality of Reference - Principle of Locality

- Reference: Describes a reference to memory adress for command access and read/writing operands
- Locality: Refers to relative points in time (temporal locality) at which commands and operands are accesed, as well as relative positions in the main memory (spatial locality)
- If an element has been referenced, there is an increased proability of repetition of access
- Spatial locality: If an element has been accessed, increased probability of nearby elements being accessed

Operands: A typical program contains array. They both affect temporal and spatial localities.

17.3 Cache and memory hierarchy

Objecive: Make the most use of locality

- Use caches for faster access
- **Cache hit:** The data we are searching for is in the small, fast memory (**Low Latency Access**)
- **Cache miss:** Data is in the main memory. (**High Latency Access (DRAM)**)

17.3.1 Characteristics of the Memory Hierarchy

1. Level 1: KB – (L1 Cache) – Smallest information unit: 4-8 Bytes (word)
2. Level 2: MB – (L2 Cache) – Smallest information unit: 8-32 Bytes (Block)
3. Level 3: GB – (Main Memory) – Smallest information unit: 1-4 Blocks
4. Level 4: TB – (Secondary Memory) – Smallest information unit: 1024+ Bytes (disk sector)

The cost per byte is highest at Level one and cheapest at Level 4

17.3.2 Caches

Caches use both types of predictability to store their data. Both spatial and temporal locality are used.

L1 Cache is divided in Data Cache and Instruction Cache. Both L1 and L2 caches are found in the main proccessor

Terminology: Hit and Miss Rate: A hit is defined as the data being found in the upper level memory (Block X)

Hit Rate: Amount of memory accesses in upper level memory

Hit time: Access time in upper level = Data access time + time to decide hit/miss

Miss: Data isn't found in the upper level memory and must be fetched in the lower level memory.

- Miss Rate = 1- Hit rate
- Miss Penalty: Time necessary to make a switch in the upper level memory + time to send the data block

  Hit Time ¡¡¡ Miss Penalty

17.3.3 Direct Mapped Cache

Its a method to find a memory address for data in the cache. This method directly uses the address of the word in main memory

Each storage address maps to an cache address.

Address assignment

- Cache Index: Memory Address mod Nº blocks in cache
- Cache Tag: Memory Address without the index digits

17.3.4 Fully Associative Cache

Each memory location can be mapped to any of the eight addresses within the cache.

The tag describes the complete word address in the main memory

Searching the cache for tags to locate a data word can be expensive

17.3.5 Associative Memory for 4-Bit tags

STILL TODO. NOT UNDERSTANDING EVERYTHING